

LDD ou DDL

Langage de Définition des Données

LDD

- ❑ En anglais **DDL** (Data Definition Language)
- ❑ Langage sous-ensemble du **SQL** (Structured Query Language)
- ❑ **Permet la manipulation des structures des données et non les données:**
 - ❑ Définition du domaine des données (ensemble des valeurs possibles)
 - ❑ Création des Tables et définitions de leurs colonnes
 - ❑ Définition des relations entre les Tables (contraintes de FK)
 - ❑ Définition de règles sur les valeurs des données (contraintes)

Les commandes de base

- ❑ **CREATE**: Création d'un élément ou d'une structure de données
- ❑ **ALTER**: Modification d'un élément ou d'une structure de données
- ❑ **DROP**: Suppression d'un élément ou d'une structure de données
- ❑ **RENAME**: Renommage d'un élément ou d'une structure de données

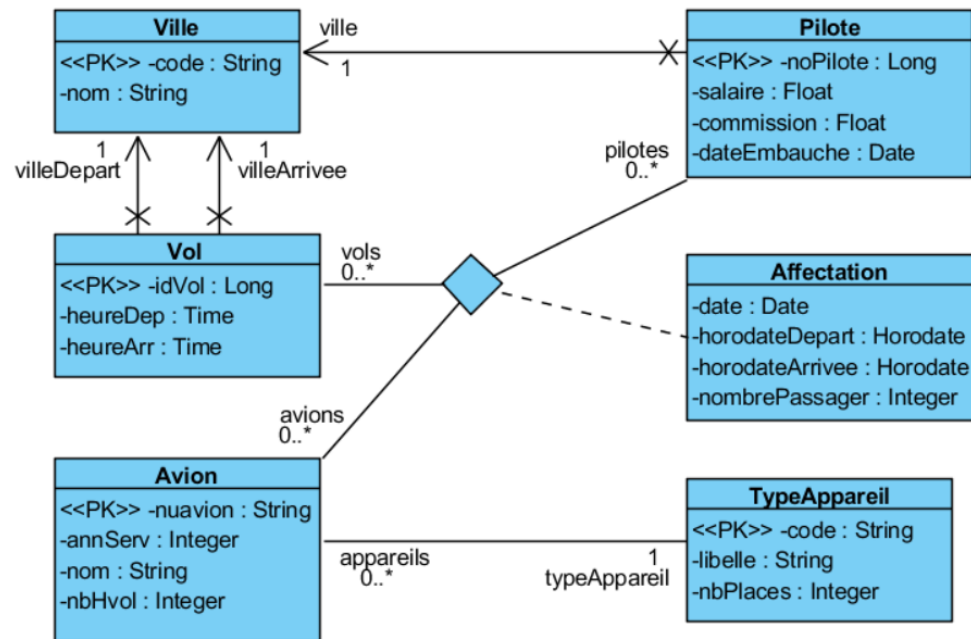
Éléments ou structures de données cibles

- ❑ **DATABASE**: base de données
- ❑ **TABLE**: table ou relation
- ❑ **INDEX**: index composé de une à plusieurs colonnes sur une table
- ❑ **VIEW**: vue, table virtuelle
- ❑ **SEQUENCE**: série ou suite de nombres
- ❑ **USER**: utilisateur auquel on pourra affecter des droits
- ❑ **ROLE**: privilèges donnés à un utilisateur

LDD par l'exemple



Le Modèle logique relationnel de la base de données « **GestionVol** »



Vol(idVol, fk_vildep, fk_vilarr, heureDep, heureArr)

Avion(nuavion, annServ, nom, nbHvol, fk_codeTypeAppareil)

TypeAppareil(code, typeAppareil, nbPlaces)

Pilote(nopilote, fk_ville, salaire, commission, dateEmbauche)

VolRealise(idVol, nuavion, nopilote, date, nbPassager)

Ville(code, nom)

DATABASE

❏ Syntaxe : Pour créer, modifier et supprimer une base de données

CREATE | ALTER | DROP DATABASE *<database>*
[**CHARACTER SET** *<charset>*]
[**COLLATE** *<collation>*]
[**TABLESPACE** *<nom table space>*]
[...]

```
CREATE DATABASE gestionvol_2021
WITH
OWNER = admin
ENCODING = 'UTF8'
LC_COLLATE = 'French_France.1252'
```

TABLE (1)

❑ Création des tables avec leurs colonnes, leurs clés primaires et étrangères

----- Table Ville (script condensé) -----

```
CREATE TABLE ville (  
    code CHAR(5) PRIMARY KEY,  
    nom VARCHAR(40)  
);
```

----- Table Vol-----

```
CREATE TABLE vol (  
    idvol SERIAL PRIMARY KEY,  
    fk_vildep CHAR(5) REFERENCES ville(code) NOT NULL,  
    fk_vilarr CHAR(5) REFERENCES ville(code) NOT NULL,  
    heuredep TIME NOT NULL,  
    heurearr TIME NOT NULL  
);
```

----- Table Pilote-----

```
CREATE TABLE pilote (  
    nopilote SERIAL PRIMARY KEY,  
    fk_ville CHAR(5) REFERENCES ville(code) NOT NULL, salaire  
    money NOT NULL, commission money NOT NULL default 0,  
    dateembauche TIMESTAMP NOT NULL  
);
```

-- Type énuméré « etat » pour les avions

```
CREATE TYPE public.etat AS ENUM ('NEUF', 'USE', 'AU_REBUT');
```

----- Table Ville (script complet équivalent) -----

```
CREATE TABLE public.ville  
(  
    code character(5) COLLATE pg_catalog."default" NOT NULL,  
    nom character varying(40) COLLATE pg_catalog."default",  
    CONSTRAINT ville_pkey PRIMARY KEY (code)  
)
```

----- Table Vol-----

```
CREATE TABLE public.vol  
(  
    idvol integer NOT NULL DEFAULT nextval('vol_idvol_seq'::regclass),  
    fk_vildep character(5) COLLATE pg_catalog."default" NOT NULL,  
    fk_vilarr character(5) COLLATE pg_catalog."default" NOT NULL,  
    heuredep time without time zone NOT NULL,  
    heurearr time without time zone NOT NULL,  
    CONSTRAINT vol_pkey PRIMARY KEY (idvol),  
    CONSTRAINT vol_fk_vilarr_fkey FOREIGN KEY (fk_vilarr)  
        REFERENCES public.ville (code) MATCH SIMPLE  
    CONSTRAINT vol_fk_vildep_fkey FOREIGN KEY (fk_vildep)  
        REFERENCES public.ville (code) MATCH SIMPLE  
)
```

TABLE (2)

----- Table TypeAppareil-----

```
CREATE TABLE typeappareil (  
    code CHAR(5) PRIMARY KEY,  
    typeappareil VARCHAR(40),  
    nbplaces SMALLINT  
);
```

----- Table Avion-----

```
CREATE TABLE avion (  
    nuavion CHAR(5) PRIMARY KEY,  
    annserv SMALLINT NOT NULL,  
    nom VARCHAR(40) UNIQUE,  
    nbhvol INTEGER,  
    etatusure etat NOT NULL,  
    fk_codeTypeAppareil CHAR(5) REFERENCES typeappareil (code) NOT NULL  
);
```

----- Table VolRealise-----

```
CREATE TABLE volrealise (  
    idvol INTEGER REFERENCES vol (idvol) NOT NULL,  
    nuavion CHAR(5) REFERENCES avion (nuavion) NOT NULL,  
    nopilote INTEGER REFERENCES pilote (nopilote) NOT NULL,  
    datevol DATE, nbpassager SMALLINT,  
    CONSTRAINT volrealise_pkey PRIMARY KEY (idvol, nuavion, nopilote, datevol)  
);
```

Script complet équivalent :

----- Table Typeappareil-----

```
CREATE TABLE public.typeappareil  
(  
    code character(5) COLLATE pg_catalog."default" NOT NULL,  
    typeappareil character varying(40) COLLATE pg_catalog."default",  
    nbplaces smallint,  
    CONSTRAINT typeappareil_pkey PRIMARY KEY (code)  
);
```

----- Table Avion-----

```
CREATE TABLE public.avion  
(  
    nuavion character(5) COLLATE pg_catalog."default" NOT NULL,  
    annserv smallint NOT NULL,  
    nom character varying(40) COLLATE pg_catalog."default",  
    nbhvol integer,  
    etatusure etat NOT NULL,  
    fk_codetypeappareil character(5) COLLATE pg_catalog."default" NOT NULL,  
    CONSTRAINT avion_pkey PRIMARY KEY (nuavion),  
    CONSTRAINT avion_nom_key UNIQUE (nom),  
    CONSTRAINT avion_fk_codetypeappareil_fkey FOREIGN KEY (fk_codetypeappareil)  
        REFERENCES public.typeappareil (code)  
);
```


TABLE (3)

- ❑ Suppression d'une colonne

```
ALTER TABLE avion DROP COLUMN nbhvol;
```

- ❑ Ajout et modification d'une colonne

```
ALTER TABLE avion ADD COLUMN nbhvol INTEGER;
```

```
ALTER TABLE avion ALTER COLUMN nbhvol SET NOT NULL;
```

CONSTRAINT

- ❑ Les Contraintes sont appliquées sur une ou plusieurs colonnes:
 - ❑ **Clé primaire :**
`CONSTRAINT avion_pkey PRIMARY KEY (nuavion)`
 - ❑ **Clé étrangère :**
`CONSTRAINT avion_fk_codetypeappareil_fkey FOREIGN KEY (fk_codetypeappareil)
REFERENCES public.typeappareil (code) MATCH SIMPLE`
 - ❑ **Valeur unique :**
`CONSTRAINT avion_nom_unique UNIQUE (nom)`
 - ❑ **Contrôle par une expression :**
`CONSTRAINT appareil_nbplace_cherck CHECK (nbplace >= 0)`

INDEX

❑ Les index créés sur une ou plusieurs colonnes permettent d'accélérer les sélections, les tris et les jointures sur une table

❑ **Création index sur le code du type d'appareil :**

```
CREATE INDEX idx_typeappareil_code ON appareil USING btree (code)
```

❑ **Création index sur l'année de mise en service de l'avion et son nom :**

```
CREATE INDEX idx_avion_annee_nom ON avion USING btree (annserv, nom)
```

VIEW₍₁₎

- ❑ Les vues sont des tables virtuelles obtenues par l'agrégation de données des tables de la BDD.
- ❑ Dans certains SGBD (comme Oracle) les vues ne sont pas virtuelles, elles sont stockées sur disque et deviennent alors un cache utilisé comme une table.
- ❑ Les vues permettent:
 - ❑ De nommer une requête complexe pour pouvoir la réutiliser.
 - ❑ De présenter des données aux utilisateurs en occultant la complexité du schéma de la BDD
 - ❑ De masquer certaines données à certains utilisateurs.

VIEW₍₂₎

Création d'une vue:

```
CREATE VIEW vue_affectation AS  
SELECT *  
FROM avion  
INNER JOIN typeappareil ON avion.fk_codetypeappareil = typeappareil.code;
```

METABASE

❑ En utilisant le LDD nous avons défini une BDD comportant des Tables avec leurs champs et leurs contraintes.

❑ Mais où sont stockées ces informations ?

Réponse: Dans une BDD particulière gérée par le SGBD que l'on nomme « METABASE »

<https://www.postgresql.org/docs/9.1/static/catalogs.html>